WEBLAB / PROGRAMAÇÃO WEB

Formulários para a web

Caderno do estudante

Professor Fabrício Rodrigues

Curso Técnico em Informática

16 de outubro a 18 de dezembro de 2026

Explicações, exemplos, atividades e mapas. Os trechos de código são pistas de estudo, não substituem o arquivo completo da prática.

9 encontros. Avaliação em 11/12. Devolutiva e recuperação em 18/12. Sem aula em 20/11.

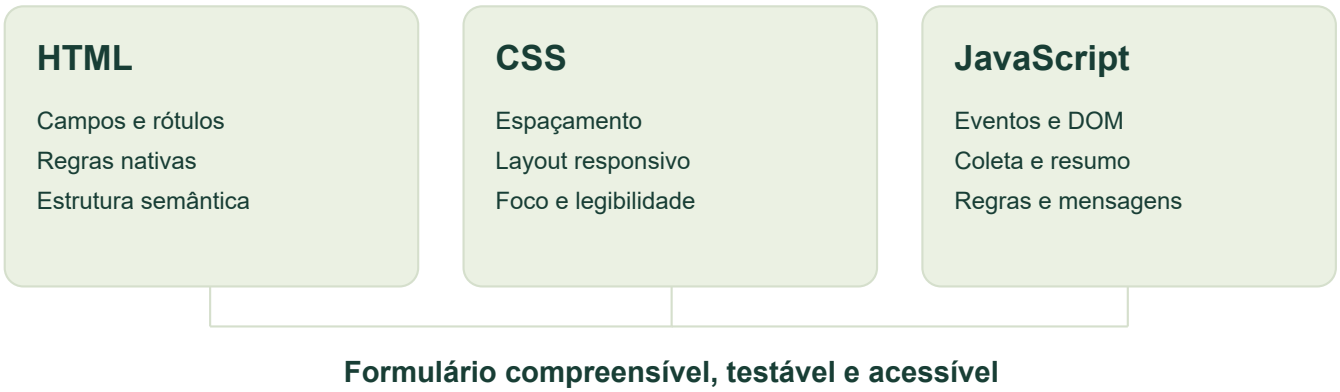
Como usar este caderno

Antes da aula, leia o objetivo e a retomada. Durante a prática, altere um comportamento por vez e registre o teste. Depois, use o checklist para explicar o que conseguiu construir.

O projeto do período

Vamos desenvolver um chamado de suporte ao laboratório com dados fictícios. A versão final coleta informações, verifica regras e apresenta um resumo local. O projeto informa que o envio é simulado.

Mapa dos conhecimentos



Arquivos para estudo sem internet

Baixe Diagnostico-base.zip ou Formulario-base.zip no WebLab e extraia o arquivo. No VS Code, use Arquivo > Abrir Pasta e selecione a pasta extraída. Abra index.html, clique com o botão direito e escolha Open with Live Server. Salve com Ctrl+S e confira o navegador. No pacote completo, as pastas estão em pratica/diagnostico-base e pratica/formulario-base. O modelo permite consultar os comportamentos após sua tentativa.

O PDF pode ser lido ou impresso. Os links das referências exigem internet quando forem abertos. Os projetos não exigem pacotes npm. Para o fluxo da turma, tenha VS Code e a extensão Live Server já instalados. Entregue o diagnóstico em TXT identificado e as práticas em ZIP com código e registro de testes, durante a aula.

Mapa do período

Todas as datas abaixo são sextas-feiras de 2026. Avaliação principal em 11/12 e devolutiva, correções e recuperação em 18/12.

Data	Tema	Evidência da aprendizagem
16/10	O que já sabemos sobre a web?	TXT identificado, projeto de diagnóstico em ZIP com registro de testes e primeiro formulário com dois campos.
23/10	Campos com significado	Versão 1 do formulário com campos e restrições HTML.
30/10	Um formulário legível em qualquer tela	Versão 2 com layout responsivo e teste de teclado.
06/11	Os dados aparecem na página	Versão 3 que apresenta um resumo sem recarregar a página.
13/11	Mensagens que ajudam a corrigir	Versão 4 com uma regra de confirmação de contato e mensagens claras.
27/11	O caminho dos dados	Especificação curta do projeto e mapa do percurso dos dados.
04/12	Um formulário que reage ao contexto	Versão integradora com campo complementar condicional.
11/12	Avaliação prática do formulário	Projeto em ZIP, registro de seis testes e explicação individual entregue em 11/12.
18/12	Corrigir, recuperar e concluir	Versão corrigida, registro das melhorias e recuperação, quando necessária, em 18/12.

20/11: sem aula. A retomada acontece em 27/11.

ENCONTRO 01 / 16/10 / Diagnóstico

O que já sabemos sobre a web?

Diagnóstico de HTML, CSS e JavaScript e fundamentos de formulários

O que você deverá conseguir

• Ler trechos de HTML, CSS e JavaScript e explicar o comportamento esperado.

• Localizar um erro de seletor e registrar como o corrigiu.

• Explicar para que serve um formulário e relacionar uma informação a um controle adequado.

• Associar um rótulo a um campo no primeiro formulário.

Ponto de partida

O portal anterior apresenta semântica HTML, Flexbox/Grid, responsividade, DOM, eventos e login simulado. A presença no material indica conteúdos trabalhados, e o diagnóstico verificará o domínio real da turma.

Diagnóstico como ponto de partida

A atividade inicial serve para orientar as próximas aulas. Registre o que consegue explicar sozinho, o que faz com uma pista e o que ainda precisa estudar. Uma resposta incompleta com raciocínio ajuda o professor a planejar a retomada.

Responda às nove questões antes da conversa coletiva. Na prática, faça pequenas mudanças em uma página conhecida. Depois da tentativa individual, a turma discutirá as estratégias e as dúvidas.

Uma página conhecida, um problema por vez

Na base de diagnóstico, a classe do destaque e os seletores do JavaScript não correspondem ao HTML. Compare os nomes literalmente, leia o console e teste a página após cada alteração. Acrescente um cartão e explique por que escolheu as tags usadas.

Fundamentos: o que é um formulário?

Um formulário é uma interface que permite informar dados e fazer escolhas para realizar uma tarefa. Em vez de apenas ler uma página, a pessoa participa: procura um conteúdo, responde a uma pesquisa ou solicita atendimento.

Antes do código, defina a finalidade, quem vai preencher, quais dados são necessários e qual resultado a pessoa espera. Um bom formulário pede somente o que ajuda a cumprir essa finalidade.

Converse com a turma: qual foi o último formulário que você usou? O que precisou informar e o que aconteceu depois?

Tipos de formulário conforme a finalidade

Os nomes abaixo descrevem usos comuns. Cadastro, acesso e pesquisa não são valores do atributo type do HTML; cada formulário combina campos conforme sua finalidade.

Finalidade	Exemplo de informação e resultado
Cadastro ou inscrição	Informar dados para participar de uma atividade.

Finalidade	Exemplo de informação e resultado
Acesso (login)	Apresentar credenciais para solicitar acesso a uma conta.
Pesquisa ou avaliação	Registrar opiniões, respostas ou preferências.
Contato ou suporte	Descrever uma dúvida ou um problema para solicitar atendimento.
Busca ou filtro	Informar um termo ou escolher critérios para encontrar resultados.

Tipos de campo: escolher pelo dado

O controle deve combinar com a informação solicitada. input possui vários tipos definidos pelo atributo type. textarea e select são elementos próprios.

Neste primeiro contato, reconheça as possibilidades. Na prática de hoje, usaremos apenas texto curto para equipamento e texto longo para descrição. Os demais controles serão construídos em 23/10.

Informação desejada	Controle HTML e exemplo
Texto em uma linha	input type="text": código PC-07. Para contato, email; para senha, password.
Descrição em várias linhas	textarea: relato do problema.
Uma opção do grupo	input type="radio" com o mesmo name, ou select: prioridade ou categoria.
Opções independentes	input type="checkbox": sintomas observados.
Data ou quantidade	input type="date" ou type="number", conforme o dado. Um código como PC-07 continua sendo texto.
Escolha de arquivo	input type="file": selecionar um anexo. Selecionar não significa enviar.

Mapa: do preenchimento ao resultado

Pessoa preenche ou escolhe → navegador confere as regras definidas → pessoa solicita a ação → sistema trata os dados → interface informa o resultado.

No projeto da turma, o JavaScript trata a ação no próprio navegador e o envio é simulado. A versão final mostrará um resumo local. Em um sistema conectado, uma requisição pode levar os dados a um servidor, que também deve validá-los e devolver uma resposta.

Preencher, validar, enviar e armazenar são etapas diferentes. Uma mensagem escrita na página só comprova o resultado que o sistema realmente verificou. A escolha de armazenar depende da finalidade e da implementação.

Antes de abrir o código: para um chamado de suporte, qual campo serve para PC-07? Qual serve para explicar o defeito? Que informação seria desnecessária? Justifique suas escolhas.

A ponte para formulários

Um chamado de suporte precisa receber informações que uma página de divulgação apenas exibe. Começaremos com equipamento e descrição do problema. Os dados do exercício são fictícios e ficam no navegador.

O label descreve o campo. Seu atributo for deve apontar para o id do controle. O name será a chave usada na coleta dos valores.

```
<form id="chamado">
  <label for="equipamento">Equipamento</label>
  <input id="equipamento" name="equipamento">
  <label for="descricao">Descrição do problema</label>
  <textarea id="descricao" name="descricao"></textarea>
  <button type="button">Protótipo em construção</button>
</form>
```

Atividade da aula

1. Abrir a ficha digital, informar nome e turma e responder individualmente. Baixar o TXT e entregar o arquivo ao professor durante a aula.

2. Baixar Diagnostico-base.zip, extrair e abrir a pasta no VS Code. Iniciar index.html com Open with Live Server. Corrigir a classe CSS e o seletor JavaScript, incluir um cartão e testar dois cliques.

3. Antes de abrir o formulário, identificar sua finalidade e justificar os campos para equipamento e descrição.

4. Baixar Formulario-base.zip em outra pasta. Abrir no VS Code com Live Server, localizar os dois campos, alterar um rótulo e testar sua associação.

5. Registrar uma facilidade e uma dúvida para a próxima aula.

Checklist de conferência

[ ] Expliquei uma decisão de HTML, uma de CSS e uma de JavaScript.

[ ] Registrei o erro que encontrei e como testei a correção.

[ ] Expliquei a finalidade do chamado e por que equipamento usa texto e descrição usa textarea.

[ ] Associei cada label ao id do campo.

Uma descoberta útil

Clicar em um rótulo associado pode levar o foco ao campo. Essa associação também ajuda tecnologias assistivas.

Para ir além

Acrescentar um campo de contato fictício e justificar o tipo escolhido.

Referências da aula

Base anterior: portal do Prof. Ábner Lucas

MDN: seu primeiro formulário

W3C WAI: rótulos dos controles

Live Server de Ritwick Dey: abrir o projeto (em inglês)

MDN: controles nativos de formulário (em inglês)

MDN: tipos de input

MDN: envio de dados

ENCONTRO 02 / 23/10 / HTML

Campos com significado

Estrutura, rótulos e regras nativas do formulário

O que você deverá conseguir

• Distinguir id, name, type e value.

• Escolher controles adequados a texto, escolha única e múltiplas opções.

• Usar required e restrições coerentes com o dado solicitado.

Ponto de partida

Retomar as dificuldades de HTML observadas em 16/10 e a visão geral dos fundamentos de formulários. Agora construiremos os controles apresentados, com rótulos, names e regras coerentes.

Um campo tem mais de um nome

id identifica um elemento na página e conecta o label. name identifica o dado coletado. type define o comportamento do controle, e value representa seu valor. Os ids devem ser únicos no documento.

Use rótulos visíveis. Um placeholder pode dar um exemplo curto, mas desaparece durante o preenchimento e não substitui o label.

```
<label for="contato">E-mail de contato fictício</label>
<input id="contato" name="contato" type="email" required>
```

Controles adequados ao problema

Use input text para um código de equipamento como PC-07. Esse código não é uma quantidade para calcular. Use select para escolher uma categoria e textarea para descrever o defeito.

Radio permite uma escolha em um grupo com o mesmo name. Checkbox permite marcar opções independentes. Agrupe opções relacionadas em fieldset, com legend que explique o conjunto.

```
<fieldset>
  <legend>Prioridade do chamado</legend>
  <label><input type="radio" name="prioridade" value="normal" required>
    Normal</label>
  <label><input type="radio" name="prioridade" value="alta"> Alta</label>
</fieldset>
```

Regras que o navegador já conhece

required pede um valor. type=email confere um formato de e-mail, sem provar que a caixa existe. minlength e maxlength delimitam comprimentos. min e max se aplicam, por exemplo, a quantidades numéricas.

Explique a regra antes do erro. Se a descrição precisa de pelo menos 20 caracteres, apresente essa orientação junto ao campo. Comece com validação nativa e teste o formulário sem JavaScript.

```
<label for="descricao">Descrição do problema</label>
<p id="ajuda">Escreva pelo menos 20 caracteres.</p>
<textarea id="descricao" name="descricao" required
  minlength="20" maxlength="500" aria-describedby="ajuda"></textarea>
```

Atividade da aula

1. Definir quais dados são necessários a um chamado fictício.
2. Criar campos de solicitante fictício, contato, equipamento, categoria, prioridade e descrição.
3. Adicionar duas opções independentes em checkbox para sintomas observados.
4. Testar campos vazios, e-mail sem formato adequado e descrição curta.

Checklist de conferência

[ ] Todos os campos têm rótulo e name.

[ ] Os ids são únicos e correspondem aos labels.

[ ] Radio e checkbox representam escolhas adequadas.

[ ] As restrições têm instruções visíveis.

Uma descoberta útil

Um button dentro de um formulário pode atuar como envio quando o tipo não é especificado. Declarar type evita ações acidentais.

Para ir além

Criar uma pequena tabela justificando tipo, name e regra de cada campo.

Referências da aula

MDN: tipos de input

W3C WAI: rótulos dos controles

W3C WAI: agrupamento de controles

MDN: validação de formulários

MDN: elemento button

Live Server de Ritwick Dey: abrir o projeto (em inglês)

ENCONTRO 03 / 30/10 / CSS e acessibilidade

Um formulário legível em qualquer tela

Layout, foco, instruções e navegação pelo teclado

O que você deverá conseguir

• Organizar os campos sem prejudicar a ordem de leitura.

• Adaptar o layout ao celular e ao computador.

• Manter foco visível e instruções compreensíveis.

Ponto de partida

Reaplicar o box model e o CSS Grid apresentados no trabalho anterior. Os campos continuam sendo controles HTML nativos.

A ordem começa no HTML

Organize os campos em uma sequência que faça sentido antes de criar colunas. A leitura no celular e a ordem da tecla Tab devem acompanhar essa sequência. O CSS pode mudar a aparência, mas não deve tornar a navegação confusa.

Use largura fluida, espaçamento consistente e fonte legível. Evite definir altura fixa para textarea quando isso impedir a leitura.

```
* { box-sizing: border-box; }
input, select, textarea, button { font: inherit; }
input:not([type="radio"]):not([type="checkbox"]),
select, textarea { width: 100%; padding: .7rem; }
.campo { display: grid; gap: .35rem; }
```

Colunas quando houver espaço

Comece com uma coluna. Adicione duas colunas em telas maiores e mantenha a descrição ocupando a largura necessária. Teste com a janela estreita e com ampliação de texto, em vez de confiar apenas em uma captura de tela.

```
.campos { display: grid; gap: 1rem; }
@media (min-width: 48rem) {
  .campos { grid-template-columns: 1fr 1fr; }
  .campo-largo { grid-column: 1 / -1; }
}
```

Pistas que não dependem só da cor

A borda vermelha, sozinha, não explica um erro. Escreva o que precisa mudar e mantenha a orientação próxima do campo. Não apague o contorno de foco sem oferecer uma alternativa visível.

Navegue com Tab e Shift+Tab, acione os controles pelo teclado e clique nos labels. Confira se textos continuam legíveis e se a página evita rolagem horizontal.

```
:focus-visible {
  outline: 3px solid #54753a;
  outline-offset: 3px;
}
.ajuda { color: #52634d; font-size: .95rem; }
```

Atividade da aula

1. Aplicar uma identidade visual própria ao formulário existente.

2. Usar uma coluna no celular e duas onde houver espaço.

3. Manter a descrição em uma área ampla e os labels próximos dos campos.

4. Trocar o computador com um colega e testar apenas pelo teclado.

Checklist de conferência

[ ] O formulário funciona em 360px e em uma tela larga.

[ ] A tecla Tab segue uma ordem compreensível.

[ ] O foco é visível em links, botões e campos.

[ ] As instruções não dependem apenas da cor.

Uma descoberta útil

A ordem visual pode mudar com CSS sem mudar a ordem de foco. Isso pode confundir quem navega usando o teclado.

Para ir além

Testar o navegador com zoom de 200% e documentar uma melhoria.

Referências da aula

MDN: estilização de formulários

MDN: fundamentos de CSS Grid

W3C WAI: rótulos dos controles

W3C WAI: notificações e erros

Live Server de Ritwick Dey: abrir o projeto (em inglês)

ENCONTRO 04 / 06/11 / JavaScript

Os dados aparecem na página

Evento submit, FormData e resumo da solicitação

O que você deverá conseguir

• Tratar o evento submit no formulário.

• Ler os valores por meio de FormData e names coerentes.

• Atualizar texto no DOM sem interpretar a entrada como HTML.

Ponto de partida

Os eventos de clique e a manipulação do DOM já aparecem no acervo anterior. Agora o evento relevante pertence ao formulário.

O evento pertence ao formulário

Escute submit no form para contemplar o envio pelo botão e pelo teclado. event.preventDefault() cancela a ação padrão desse envio. Ele não salva os dados e não cria um servidor.

Na simulação, o JavaScript mostra um resumo local. Uma mensagem deve dizer que o chamado não foi enviado a um serviço real.

```
const form = document.querySelector("#chamado");
const resumo = document.querySelector("#resumo");
form.addEventListener("submit", (event) => {
  event.preventDefault();
  const dados = new FormData(form);
  resumo.textContent = `Equipamento: ${dados.get("equipamento")}`;
});
```

As chaves vêm do name

new FormData(form) coleta os controles elegíveis. Um campo sem name não aparece como uma entrada com a chave esperada. Checkbox desmarcado e controle disabled não entram normalmente nessa coleta.

Use get para um valor e getAll quando vários controles compartilham o mesmo name, como uma lista de sintomas. Uma string pode receber trim para retirar espaços no começo e no fim.

```
const dados = new FormData(form);
const descricao = String(dados.get("descricao") || "").trim();
const sintomas = dados.getAll("sintomas");
console.log(descricao, sintomas);
```

Um resumo que continua sendo texto

Use textContent para exibir os valores digitados. Assim, uma entrada parecida com uma tag aparece como texto. Evite construir HTML diretamente com o conteúdo preenchido pelo visitante.

Uma região de status pode avisar que o resumo mudou. O teste precisa verificar o conteúdo exibido, o caminho pelo teclado e a ausência de erros no console.

```
<p id="resumo" role="status" aria-live="polite"></p>
```

Atividade da aula

1. Escutar submit com addEventListener no form.
2. Coletar equipamento, categoria, prioridade e descrição.
3. Mostrar um resumo com textContent e aviso de simulação.
4. Testar envio pelo botão e pela tecla Enter em um campo apropriado.

Checklist de conferência

[ ] O envio não recarrega a página na simulação.

[ ] FormData encontra os names definidos no HTML.

[ ] Os checkbox usam getAll quando necessário.

[ ] O resumo apresenta os dados como texto e informa a simulação.

Uma descoberta útil

Um campo pode ter id e aparecer na tela, mas não ser coletado pela chave esperada quando falta name.

Para ir além

Acrescentar um contador de caracteres com evento input, mantendo a regra minlength no HTML.

Referências da aula

MDN: evento submit

MDN: uso de FormData

MDN: atributo disabled

MDN: textContent

Live Server de Ritwick Dey: abrir o projeto (em inglês)

ENCONTRO 05 / 13/11 / Validação

Mensagens que ajudam a corrigir

Regras complementares, erros e casos de teste

O que você deverá conseguir

• Distinguir restrição HTML e regra complementar em JavaScript.

• Limpar uma mensagem personalizada quando o valor muda.

• Testar entradas válidas, inválidas e limites.

Ponto de partida

Retomar required, minlength e type=email. A regra nova compara dois campos, sem substituir as regras que o HTML já oferece.

Uma regra que compara dois campos

Acrescente um campo para repetir o contato fictício. A confirmação deve coincidir com o primeiro contato. Deixe a regra explícita e apresente a ajuda antes da tentativa de envio.

O exemplo trabalha uma comparação simples de strings. A atividade usa dados fictícios, sem senhas ou documentos pessoais. A validação do navegador ajuda o preenchimento, mas um sistema real também verifica os dados no servidor.

Mensagem personalizada precisa ser limpa

setCustomValidity recebe uma mensagem quando o valor não atende à regra. Uma string vazia limpa o erro personalizado. Atualize a regra quando qualquer um dos dois contatos mudar para não manter um erro antigo.

A validação nativa pode impedir o evento submit quando há um campo inválido. Por isso, preparar e atualizar a regra no evento input evita depender apenas do submit.

```
const contato = document.querySelector("#contato");
const confirmar = document.querySelector("#confirmar");
function conferirContato() {
  const diferente = confirmar.value.trim() !== contato.value.trim();
  confirmar.setCustomValidity(diferente ? "Repita o mesmo contato." : "");
}
contato.addEventListener("input", conferirContato);
confirmar.addEventListener("input", conferirContato);
```

Teste a regra e a recuperação

Verifique campo vazio, formato inválido, valores diferentes e correção depois de uma falha. O formulário deve permitir nova tentativa sem exigir que a pessoa reescreva os campos corretos.

Registre entrada, resultado esperado, resultado observado e ajuste realizado. Uma descrição de erro deve explicar o próximo passo, e não apenas dizer “inválido”.

```
if (!form.reportValidity()) {
  return;
}
// Depois de validar, gerar o resumo local.
```

Atividade da aula

1. Criar a confirmação do contato fictício com label e name.
2. Comparar os campos e limpar o erro personalizado após a correção.
3. Exibir uma orientação legível junto ao campo.
4. Registrar pelo menos quatro testes, incluindo a recuperação do erro.

Checklist de conferência

[ ] As restrições HTML continuam funcionando.

[ ] A regra de comparação atualiza quando os valores mudam.

[ ] O erro desaparece depois da correção.

[ ] Meus testes incluem ao menos um caso inválido e sua recuperação.

Uma descoberta útil

Uma mensagem personalizada não vazia mantém o controle inválido. Alterar o texto do campo, sozinho, não apaga a mensagem definida pelo código.

Para ir além

Relacionar uma mensagem própria ao campo por aria-describedby e testar sua clareza com um colega.

Referências da aula

MDN: validação de formulários

MDN: setCustomValidity

W3C WAI: notificações e erros

MDN: evento submit

Live Server de Ritwick Dey: abrir o projeto (em inglês)

ENCONTRO 06 / 27/11 / Web e projeto

O caminho dos dados

GET, POST, HTTPS e definição do projeto integrador

O que você deverá conseguir

• Explicar cliente, requisição e servidor em linguagem simples.

• Distinguir parâmetros de GET e corpo de POST.

• Definir o que o protótipo local faz e o que exigiria backend.

Ponto de partida

Não haverá aula em 20/11. Reabrir a versão anterior e conferir o resumo antes de apresentar conceitos novos.

Uma requisição precisa de destino

O navegador pode enviar um formulário a uma URL indicada por action. method define como essa requisição se organiza. Em GET, os campos enviados aparecem normalmente na consulta da URL. Em POST, os dados ficam no corpo da requisição.

POST não significa criptografia. HTTPS protege o tráfego entre os pontos da conexão. Um serviço real precisa receber, validar e tratar os dados.

```
<!-- Exemplo conceitual: esta rota não existe no WebLab. -->
<form action="/api/chamados" method="post">
  <!-- controles com name -->
</form>
```

O limite do protótipo está explícito

GitHub Pages hospeda conteúdo estático. O nosso projeto usa HTML, CSS e JavaScript no navegador e simula o envio. Publicar esses arquivos não cria automaticamente uma API, um banco de dados ou uma caixa de chamados.

Um backend futuro receberia a requisição, faria novas validações, aplicaria regras e retornaria uma resposta. Esse desenvolvimento fica fora das 15 horas deste percurso.

Requisitos do projeto integrador

O produto final será um formulário de suporte para um laboratório fictício. Ele deve organizar campos, orientar o preenchimento, validar entradas e mostrar um resumo local acessível.

Defina uma categoria de problema que abra um campo complementar, como “outro”. Escreva seus casos de teste e as partes que cada aluno consegue explicar, mesmo quando houver colaboração.

Atividade da aula

1. Desenhar o percurso do preenchimento ao resumo local.
2. Marcar no mapa onde entraria um servidor real.
3. Escrever os requisitos do projeto integrador e os campos necessários.
4. Escolher seis casos de teste e planejar as melhorias restantes.

Checklist de conferência

[ ] Expliquei uma diferença entre GET e POST.

[ ] Reconheci que POST sozinho não cifra dados.

[ ] Meu projeto declara envio simulado.

[ ] Os requisitos descrevem comportamentos que consigo testar.

Uma descoberta útil

Um endereço com parâmetros de busca pode ser compartilhado como link. Esse é um uso comum de GET em pesquisas.

Para ir além

Inspecionar uma requisição de busca pública nas ferramentas do navegador, sem enviar dados pessoais.

Referências da aula

MDN: envio de dados

GitHub: funcionamento do Pages

MDN: seu primeiro formulário

Live Server de Ritwick Dey: abrir o projeto (em inglês)

ENCONTRO 07 / 04/12 / Integração

Um formulário que reage ao contexto

Campos condicionais e organização do código

O que você deverá conseguir

• Mostrar um campo quando a escolha exige mais detalhes.

• Manter hidden, disabled e required coerentes.

• Organizar a lógica em funções pequenas e explicáveis.

Ponto de partida

Reaplicar alternância de estado e DOM, presentes nos exemplos anteriores de cards e detalhes.

A pergunta seguinte depende da resposta

Quando a categoria for “outro”, o formulário pede uma explicação adicional. Quando a categoria mudar, o campo complementar deve deixar de ser obrigatório e não participar da coleta.

Esconder um elemento não significa desabilitar o controle. Ajuste o estado de forma explícita. Depois de voltar a “outro”, o controle deve funcionar novamente.

```
function atualizarOutro() {
  const ativo = categoria.value === "outro";
  blocoOutro.hidden = !ativo;
  outro.disabled = !ativo;
  outro.required = ativo;
  if (!ativo) outro.value = "";
}
categoria.addEventListener("change", atualizarOutro);
atualizarOutro();
```

Funções com uma responsabilidade

Separe atualizarOutro, conferirContato e gerarResumo. Dê nomes que indiquem o que cada função faz. Isso torna mais simples localizar erros e explicar uma alteração durante a apresentação.

Ao oferecer uma ação para começar um novo chamado, limpe também o resumo e os erros personalizados. Evite apagar o trabalho em andamento por meio de um botão de reset pouco claro.

Integração se verifica com mudanças de estado

Teste “outro”, preencha detalhes, volte a uma categoria comum e gere o resumo. Depois reabra “outro” e confirme se a regra continua adequada.

Confira novamente o layout e o teclado. Um comportamento novo pode criar uma barreira de uso em algo que já funcionava.

Preparação para a avaliação de 11/12

Guarde a versão integrada e organize o README e o registro dos testes. Na aula seguinte, você entregará o projeto e explicará decisões de HTML, CSS e JavaScript. A publicação pode ser concluída depois da devolutiva; o projeto local também será avaliado.

Atividade da aula

1. Implementar a categoria “outro” e seu campo complementar.
2. Atualizar hidden, disabled e required juntos.
3. Organizar o script em funções pequenas.
4. Executar os casos de ida, volta e novo preenchimento.
5. Preparar o README e registrar os seis casos de teste para a avaliação de 11/12.

Checklist de conferência

[ ] O campo aparece apenas quando necessário.

[ ] Um campo desativado não bloqueia o envio.

[ ] A coleta reflete o estado atual do formulário.

[ ] Consigo explicar cada função do meu script.

Uma descoberta útil

Um campo com hidden no bloco pode continuar existindo no DOM. Um controle disabled normalmente fica fora de FormData.

Para ir além

Criar uma função que gere cada linha do resumo com createElement e textContent.

Referências da aula

MDN: atributo disabled

MDN: uso de FormData

MDN: evento submit

Live Server de Ritwick Dey: abrir o projeto (em inglês)

ENCONTRO 08 / 11/12 / Avaliação

Avaliação prática do formulário

Entrega do projeto, testes e explicação individual

O que você deverá conseguir

• Demonstrar o formulário com uma entrada válida, um erro e sua correção.

• Explicar decisões de HTML, CSS e JavaScript e testar uma alteração individual.

• Entregar os arquivos do projeto, o README e o registro de testes.

Ponto de partida

Usar a versão integrada de 04/12. A avaliação principal acontece em 11/12, antes do último encontro, para permitir devolutiva e correções em 18/12.

O que será entregue

Compacte a pasta do projeto com index.html, style.css, script.js, README e REGISTRO-DE-TESTES.txt. Identifique o ZIP com turma e nome. Entregue o arquivo ao professor durante a aula. O projeto deve abrir no VS Code com Live Server.

Registre seis situações: campos vazios, e-mail com formato inadequado, contatos diferentes, descrição curta, mudança para outro e retorno, e preenchimento válido. Em cada uma, escreva entrada, ação, resultado esperado e observado.

Demonstração e compreensão individual

Mostre um caso válido, provoque um erro e demonstre sua correção. Explique um rótulo ou grupo HTML, uma regra CSS e o trecho JavaScript que coleta ou valida os dados.

Faça uma pequena alteração individual orientada pelo professor e registre o teste. Todos entregam também a explicação por escrito; o professor organiza a observação nas estações conforme o tamanho da turma.

Critérios da avaliação

Rubrica proposta, total de 10 pontos: HTML e rótulos (2), CSS e acessibilidade (2), JavaScript e validação (2), testes e documentação (2), explicação e alteração individual (2). Os critérios serão apresentados antes da realização.

A hospedagem pública não é condição para demonstrar o funcionamento local. Não use dados pessoais reais. O formulário continua com envio simulado. A devolutiva e as oportunidades de correção e recuperação ficam para 18/12, conforme as regras da turma.

Atividade da aula

1. Conferir o projeto no Live Server e executar os seis casos de teste.
2. Entregar o ZIP identificado com código, README e registro de testes.
3. Demonstrar o formulário e explicar decisões de HTML, CSS e JavaScript.
4. Realizar e registrar uma pequena alteração individual solicitada pelo professor.

Checklist de conferência

[ ] O ZIP contém a pasta completa e abre no VS Code com Live Server.

[ ] Registrei resultados esperados e observados dos seis testes.

[ ] Entreguei minha explicação e a alteração individual testada.

[ ] Sei que a devolutiva e a recuperação acontecem em 18/12.

Uma descoberta útil

Um teste que registra a entrada e o resultado esperado pode ser repetido por outra pessoa e ajuda a localizar um defeito.

Para ir além

Preparar a publicação no GitHub Pages, sem substituir a entrega local e a explicação individual.

Referências da aula

MDN: validação de formulários

MDN: uso de FormData

W3C WAI: notificações e erros

Live Server de Ritwick Dey: abrir o projeto (em inglês)

ENCONTRO 09 / 18/12 / Devolutiva e recuperação

Corrigir, recuperar e concluir

Devolutiva da avaliação, recuperação e fechamento do percurso

O que você deverá conseguir

• Interpretar a devolutiva da avaliação de 11/12.

• Corrigir o projeto e demonstrar novamente os critérios que precisam de reforço.

• Comparar a aprendizagem atual com o diagnóstico inicial e organizar o portfólio.

Ponto de partida

Reabrir o projeto e a devolutiva de 11/12. A avaliação principal já foi aplicada; este encontro permite corrigir, recuperar e encerrar o percurso.

A devolutiva orienta a correção

Leia o comentário do professor e localize no código a evidência de cada ponto. Escolha as correções prioritárias e escreva o que será alterado.

Depois da mudança, repita o caso que apresentou problema e os testes relacionados. Registre o resultado para mostrar o que melhorou.

Recuperação com evidência individual

Quando necessário, refaça uma tarefa equivalente sobre os critérios que ainda precisam ser demonstrados, conforme a orientação do professor e as regras da turma. Explique a alteração e mostre o teste.

Quem já demonstrou os critérios pode aperfeiçoar acessibilidade, documentação ou publicação e ajudar a discutir estratégias, preservando a autoria individual.

Portfólio e continuidade

Compare sua explicação atual com a primeira tentativa de outubro e registre um avanço concreto. Organize a versão final e os registros de teste.

Conclua ou revise a publicação no GitHub Pages, quando houver conexão. API, fetch e validação no servidor ficam como possibilidades de continuidade.

Atividade da aula

1. Ler a devolutiva da avaliação de 11/12.

2. Corrigir os itens indicados e registrar novos testes.

3. Realizar a recuperação orientada, se necessária, e entregar a versão corrigida.

4. Organizar o portfólio, comparar com o diagnóstico e concluir a publicação quando possível.

Checklist de conferência

[ ] Relacionei a devolutiva com o meu código.

[ ] Registrei a correção e repeti os testes.

[ ] Demonstrei os critérios retomados na recuperação, quando necessária.

[ ] Guardei minha versão final e uma evidência do que aprendi.

Uma descoberta útil

Repetir o teste depois de corrigir ajuda a verificar se o problema foi resolvido e se outra parte continua funcionando.

Para ir além

Desenhar uma API futura, descrevendo os dados enviados e a resposta esperada.

Referências da aula

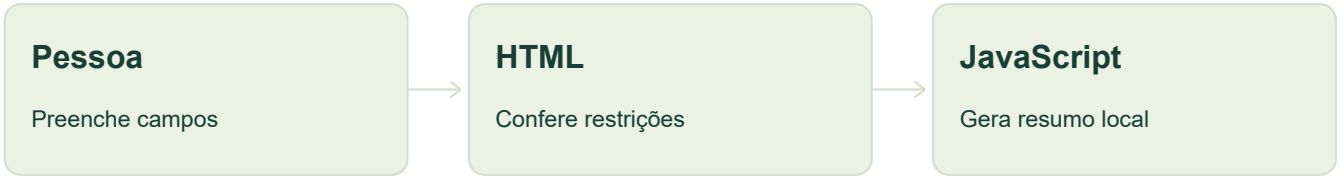
GitHub: configuração da publicação

GitHub: funcionamento do Pages

MDN: validação de formulários

Live Server de Ritwick Dey: abrir o projeto (em inglês)

Mapa do percurso dos dados



No protótipo: os dados ficam no navegador.

Em um sistema real, uma requisição iria a um servidor para nova validação e tratamento.

Servidor, banco de dados e autenticação serão temas de continuidade.

Seis testes do projeto

Caso	Resultado esperado
Campos obrigatórios vazios	O formulário orienta o preenchimento e não gera confirmação válida.
E-mail sem formato adequado	O controle indica o problema de formato.
Contatos diferentes	A confirmação pede o mesmo contato e permite correção.
Descrição curta e no limite	A regra de tamanho se comporta conforme a instrução.
Categoria outro e retorno	Campo complementar aparece, torna-se obrigatório e sai da coleta quando desativado.
Entrada válida	Resumo local mostra os valores e informa envio simulado.

Registro do teste

Para cada caso, anote: entrada utilizada, ação, resultado esperado, resultado observado e correção. Depois de corrigir, repita o teste.

Glossário de consulta

Termo	Função no projeto
DOM	Representação da página que o JavaScript consulta e altera.
id	Identifica um elemento no documento.
name	Define a chave de um dado na coleta do formulário.
label	Rótulo que identifica um controle.
submit	Evento associado à tentativa de envio do formulário.
preventDefault	Cancela a ação padrão de um evento cancelável.
FormData	Organiza entradas elegíveis de um formulário por nome.
Validação	Conferência de uma regra sobre os valores.
Backend	Parte do sistema que recebe e trata requisições em servidor.
HTTPS	Proteção do tráfego na conexão.
Responsividade	Adaptação do layout a diferentes tamanhos de tela.

Referências para continuar

Consulta das fontes em 15/10/2026. O portal anterior serve de evidência dos temas apresentados, não de comprovação do domínio individual. O texto e as práticas deste caderno são autorais.

Base anterior: portal do Prof. Ábner Lucas

MDN: seu primeiro formulário

MDN: tipos de input

W3C WAI: rótulos dos controles

W3C WAI: agrupamento de controles

MDN: validação de formulários

MDN: estilização de formulários

MDN: fundamentos de CSS Grid

MDN: evento submit

MDN: uso de FormData

MDN: setCustomValidity

W3C WAI: notificações e erros

MDN: envio de dados

MDN: elemento button

MDN: atributo disabled

GitHub: funcionamento do Pages

GitHub: configuração da publicação

MDN: textContent

VS Code: arquivos e pastas (em inglês)

Live Server de Ritwick Dey: abrir o projeto (em inglês)

MDN: controles nativos de formulário (em inglês)

Meu registro final

O que consigo fazer com autonomia? Qual evidência mostra isso? O que quero estudar na próxima etapa?

_____

_____